



BÀI TẬP NGUYÊN LÝ KIỂM THỬ PHẦN MỀM

Phan Tụ Trường - 20221889

Bài 1: Giải thích vì sao nói rằng “Kiểm thử chỉ cho thấy sự hiện diện của lỗi, chứ không chứng minh rằng phần mềm không có lỗi”. Hãy lấy ví dụ minh họa trong thực tế để làm rõ nhận định trên

- “Kiểm thử chỉ cho thấy sự hiện diện của lỗi, chứ không chứng minh phần mềm không chứng minh rằng phần mềm đó không có lỗi” nghĩa là luôn có lỗi dù có kiểm thử bao nhiêu lần. Chúng ta cần phải lên kế hoạch kiểm thử sao cho càng thấy nhiều lỗi càng tốt
- Ví dụ: Giả sử ta có 1 màn hình đăng nhập gồm tên user và password
 - Lần test 1: nhập đúng tài khoản mật khẩu, thứ ta mong chờ là thành công → đăng nhập thành công
 - Lần test 2: nhập đúng tài khoản sai mật khẩu, thứ ta mong chờ là thông báo sai mật khẩu → đăng nhập thành công
 - Lần test 3: nhập tài khoản chưa có trong hệ thống, thứ ta mong chờ là thông báo tài khoản không tồn tại → đăng nhập thành công
- Đó là lý do vì sao kiểm thử chỉ thấy sự hiện diện của lỗi và không chứng minh rằng phần mềm không có lỗi

Bài 2: Vì sao không thể kiểm thử toàn diện (Exhaustive Testing)
Hãy mô tả 2 ý do cụ thể và nêu hướng khắc phục mà nhóm kiểm thử có thể áp dụng

- Tại vì trong 1 hệ thống dù lớn hay nhỏ thì ta cũng không thể kiểm tra hoàn toàn đc tất cả các module đc vì có rất nhiều module và việc kiểm tra như thế sẽ không thể đảm bảo là ta có đủ thời gian hoàn toàn hay là đảm bảo rằng lỗi sẽ không xuất hiện nữa
- Thay vào đó chúng ta nên ưu tiên kiểm thử những phần module mà nó xử lý về mặt logic vì ở đó xem chỉ tỉ lệ cao là xảy ra lỗi và chúng ta sẽ tiết kiệm được thời gian và công sức

Bài 3: Trong một hệ thống thương mại điện tử, module “Giỏ hàng” thường phát sinh nhiều lỗi. Giải thích nguyên lý kiểm thử liên quan và cách tester nên điều chỉnh chiến lược kiểm thử.

- Trong hệ thống thương mại điện tử, module “Giỏ hàng” phát sinh lỗi thường được phân bố tập trung (Defect clustering) vì đa phần lỗi này có phần trăm cao xảy ra ở các module về xử lý logic, chức năng, hãy ưu tiên tập trung xoay tua vào những module như này trước để có thể tìm ra lỗi sớm nhất

Bài 4: Phân tích câu nói “Kiểm thử cần phụ thuộc vào bối cảnh”

Hãy nêu ví dụ minh họa kiểm thử 1 phần mềm y tế và 1 game di động, chỉ ra điểm khác nhau trong chiến lược kiểm thử

Về câu nói “Kiểm thử cần phụ thuộc vào bối cảnh” tức là ta phải có nhận định rõ rằng là không phải là phần mềm nào cũng có thể dùng nguyên dc 1 testcase mặc định vì còn phải phụ thuộc rằng là nó được khai triển như thế nào giống như việc ứng dụng di động vào ứng dụng web thì không thể áp dụng giống nhau được mà phải phụ thuộc vào chính ứng dụng đó

Ví dụ:

- Phần mềm Y tế: Ưu tiên hàng đầu là về tính chính xác của dữ liệu, tính bảo mật và độ tin cậy.
- Game di động: Trọng tâm là trải nghiệm người dùng mang tính giải trí và khả năng tương thích

Bài 5: Một nhóm phát triển phần mềm cho rằng: “Không cần kiểm thử thêm vì phần mềm không có lỗi nghiêm trọng nào”.
Bạn có đồng ý không? Giải thích dựa trên nguyên lý kiểm thử tương ứng

- Tôi không đồng ý với ý kiến trên, vì việc không kiểm thử thấy lỗi không có nghĩa là sản phẩm đã hết lỗi, mà không phải lúc nào kiểm thử cũng là để tìm ra lỗi mà dùng để kiểm thử các chức năng có đúng với yêu cầu được đề ra hay không.

Bài 6: Phân tích lợi ích và rủi ro của việc thực hiện kiểm thử sớm trong vòng đời phát triển phần mềm (Early testing) Hãy nêu ví dụ minh họa cụ thể.

Lợi ích: Chúng ta sẽ có thể tìm thấy lỗi sớm hơn giúp tiết kiệm chi phí, thời gian và công sức.

Rủi ro: Đôi khi kiểm tra sớm sẽ không thể đảm bảo chắc rằng khi hoàn thiện module hay chức năng đó không phát sinh thêm lỗi mới hay là khi ta hoàn thiện thì lỗi đó có quay lại không và xảy ra phát sinh về việc yêu cầu thay đổi liên tục.

Ví dụ:

- Về lợi ích: Như về chức năng thanh toán giả sử như nếu trong lúc mình ấn thanh toán mà bị mất kết nối internet thì nó có trừ tiền của khách hàng không, nó có tạo hóa đơn và gửi mail cho khách hàng không. Dựa vào lỗ hổng đó mà ta có thể khắc phục được lỗi sớm.
- Về rủi ro: Về chức năng tìm kiếm, ban đầu theo quy hoạch là sẽ làm 10 bộ lọc tìm kiếm, nhưng sau khi họp tình giảm đi 7 bộ lọc việc này làm cho tester bị mất thời gian để sửa lại theo yêu cầu.

Bài 7: Giải thích hiện tượng “Nghịch lý thuốc trừ sâu (Pesticide Paradox)” trong kiểm thử phần mềm

Làm thế nào để tránh rơi vào itnsh hướng trong dự án thực tế?

Về hiện tượng “Nghịch lý thuốc trừ sâu” ta có thể hiểu được là việc kiểm thử cũng như thuốc trừ sâu không phải là ta cứ dùng mãi 1 cách là có thể triệt để việc lỗi được, cần thay đổi testcase để có thể tránh việc mà sử dụng mãi 1 cách.

Ví dụ như: Ta nhập 1 bộ mẫu dữ liệu nhiều lần và nó luôn chạy tốt, hãy thử nhập 1 bộ mẫu dữ liệu khác vì biết đâu khả năng với bộ dữ liệu mới nó có thể ngẫu nhiên lỗi:

- Phương trình $a + b = x$
 - $a = 2$
 - $b = 2$
 - $x = 4$ (đúng)
- Ta thay bộ dữ liệu khác
 - $a = 3$
 - $b = 3$
 - $x = 6$ (đúng) nhưng kết quả hiện là 9(sai)

Bài 8: Hãy chọn 2 nguyên lý kiểm thử bạn cho là quan trọng nhất, giải thích lý do và nêu tình huống minh họa

2 nguyên lý kiểm thử mà cho là quan trọng nhất là: “Kiểm thử toàn bộ là không thể (Exhaustive testing is impossible) và Quan niệm sai lầm về việc “Hết lỗi” (Absence of error fallacy)”

- Kiểm thử toàn bộ là không thể (Exhaustive Testing is Impossible): Xác định giới hạn và buộc phải có chiến lược, tiết kiệm nguồn lực khổng lồ, giúp ra quyết định "Khi nào dừng lại?"
- Quan niệm sai lầm về việc “Hết lỗi” (Absence of error fallacy): Kết nối kiểm thử với mục tiêu kinh doanh, ngăn chặn việc "xây dựng sai sản phẩm", định nghĩa "Chất lượng" một cách toàn diện.

Bài 9: Bạn là trưởng nhóm kiểm thử. Hãy mô tả kịch bản thực tế trong dự án mà ít nhất 3 nguyên lý kiểm thử được thể hiện rõ

Tên dự án: "ZenShop" - Tái thiết kế quy trình thanh toán (Checkout Process) cho một nền tảng thương mại điện tử lớn.

Mục tiêu: Giảm tỷ lệ người dùng bỏ giỏ hàng (cart abandonment) bằng cách đơn giản hóa quy trình thanh toán từ 5 bước xuống còn 2 bước.

Thời gian: 3 tháng, một deadline rất gấp để kịp ra mắt cho mùa mua sắm cuối năm.

Các nguyên lý kiểm thử được thể hiện:

Kiểm thử toàn bộ là không thể (Exhaustive Testing is Impossible)

Lỗi thường tập trung lại (Defect Clustering)

Sai lầm về sự vắng mặt của lỗi (Absence-of-Errors Fallacy)

Bài 10: Hãy liên hệ 7 nguyên lý kiểm thử với một quy trình phát triển phần mềm cụ thể (Waterfall, Agile hoặc Scrum).

- Kiểm thử cho thấy sự hiện diện của lỗi: Thể hiện trong Sprint Review, khi các bên liên quan vẫn tìm ra lỗi trên chức năng được cho là "hoàn thành".
- Kiểm thử toàn bộ là không thể: Thể hiện trong Sprint Planning, khi team phải chọn lọc các kịch bản kiểm thử quan trọng nhất dựa trên rủi ro do thời gian có hạn.
- Kiểm thử sớm: Diễn ra từ khâu Backlog Refinement (làm rõ yêu cầu) và trong suốt Sprint (test song song với lập trình), giúp phát hiện lỗi từ gốc.
- Lỗi thường tập trung lại: Phát hiện qua Sprint Retrospective, khi team phân tích dữ liệu và thấy phần lớn lỗi tập trung ở một module cụ thể, từ đó có hành động cải tiến.
- Nghịch lý thuốc trừ sâu: Scrum tự nhiên chống lại điều này bằng cách yêu cầu viết kịch bản kiểm thử mới cho các User Story mới trong mỗi Sprint, giữ cho bộ test luôn được cập nhật.
- Kiểm thử phụ thuộc vào ngữ cảnh: Mỗi User Story có một ngữ cảnh riêng (bảo mật, hiệu năng, giao diện...). Do đó, chiến lược kiểm thử phải linh hoạt thay đổi theo từng story.
- Quan niệm sai lầm về việc “Hết lỗi”: Thể hiện trong Sprint Review, khi sản phẩm không có lỗi kỹ thuật nhưng không đáp ứng được nhu cầu người dùng (khó sử dụng, không hữu ích). Scrum giúp nhận ra sai lầm này sớm để kịp thời điều chỉnh.